



POLITECNICO
MILANO 1863

Machine Learning Final Report

Francesco Fossati
Niccolò Mauroner
Senura Lisan Martino Jayakody Arachchilage

July 2026

Contents

1	Matlab RL project	1
1.1	Introduction	1
1.2	RL PPO approach	2
1.3	Kinematic Environment and System Modeling	2
1.3.1	Collision Handling and Wall Boundaries	2
1.4	Formulation as Markov Decision Process (MDP)	3
1.4.1	State Space $\mathcal{S}$	3
1.4.2	Features and Dimensionality: Rationale	3
1.4.3	Action Space $\mathcal{A}$	4
1.4.4	Reward Function $\mathcal{R}$	4
1.5	Hyperparameter choice	6
1.5.1	Discount Factor $\gamma = 0.99$	6
1.5.2	GAE $\lambda = 0.95$	6
1.5.3	Weight of the Entropy Loss	7
1.5.4	Experience Horizon (2048) and Mini-Batch Size (256)	7
1.6	Test Scenarios	7
1.6.1	Training Procedure	8
1.7	Results	8
1.7.1	Learning Curve in a Sample Scenario and Models Performances	8
1.7.2	Robustness and Zero-Shot Generalization under Constant Batch Sizes	10
2	Python project	13
2.1	Dynamic Modeling of the Compliant Base Robot System	13
2.1.1	The Underactuated Reachability Challenge	14
2.2	State Estimation, Control and Parameters Optimization	15
2.2.1	LSTM versus FNN for State Estimation	15
2.2.2	LSTM Model	15
2.3	Dataset Generation and Deep Learning Training Pipeline	16
2.3.1	Input and Output Vector Formulations	16
2.3.2	Synthetic dataset generation by means of Chirp excitation	17
2.3.3	Data Normalization	17
2.3.4	Situations of Custom Training	17
2.4	Active Resonance Control	17
2.4.1	Classical velocity feedback (resonance pumping)	18
2.4.2	Model Predictive Control (MPC) Heuristic	19
2.4.3	Heuristic Dual-Phase (HDP) Policy	19

2.5	Multi-Resolution Optimization	20
2.5.1	Formulation of Optimization Space	20
2.5.2	Stability-Aware Objective Function	20
2.5.3	The 4-stage optimization algorithm	21
2.6	Results	22

Chapter 1

Matlab RL project

1.1 Introduction

Parcel singulation is an important process in automated distribution centers and is required to condition the parcel stream prior to arriving at downstream sorting and scanning devices. The goal is to achieve a highly crowded, disorganized flow of goods and to create a positive longitudinal distance between two successive parcels. This spatial separation is needed to ensure that downstream inspection sensors can identify and track each item separately and avoid overlaps and system jams due to collisions.

This project presents the design and implementation of a Reinforcement Learning (RL) controller for a multidirectional Actuator Matrix Sorting (AMS) module which consists of individual cells arranged in a 5×5 grid. Each cell controls the linear travel speed ($v \in [0.5, 2.2]$ m/s) and steering angle ($\theta \in [-\pi/4, \pi/4]$) of the parcel above it. The system is kinematically complex and physical interactions and collisions must be handled in real time between moving parcels. This makes standard model-based control methods hard to calibrate and not sufficiently reactive to parcel size and arrival rate variations. Therefore, they are excluded from the present analysis. To alleviate these issues, the agent is trained to maximize the spatial distance between successive packages measured on an outfeed treadmill downstream of the module. A stochastic policy optimization technique with Proximal Policy Optimization (PPO) is used in MATLAB. To verify the robustness and generalization capabilities of the proposed controllers, the experimental campaign is organized around **three different operational configurations** of increasing geometric and congestion complexity:

- **Square Couples (Scenario 1):** is the baseline. It involves processing ten square parcels in fixed batches of two, using a compact 75-dimensional state vector.
- **Rectangular Couples (Scenario 2):** by introducing fluctuations in aspect ratio through rectangular parcels, the state expands to 108 dimensions (including a 100-dimensional grid-state and 8 elements to monitor exited parcels) while maintaining a density of 2 parcels per batch.
- **Rectangular Batches (Scenario 3):** generating rectangular parcels in larger, random batches of 1, 2 or 3 items (10 parcels total, 112-dimensional state) increases local congestion and collision dynamics. The agent has to handle challenging multi-body contact dynamics to avoid overtaking in the sequence.

1.2 RL PPO approach

The RL controllers developed for the AMS module are based on the PPO algorithm. PPO is an actor-critic, policy-gradient method that is well suited for continuous action spaces such as the action spaces of the actuator steering angles and velocities. PPO uses a clipped surrogate objective to ensure stable policy updates, preventing the policy from changing too much in a single step. This is a key factor in physical multi-body set-ups where aggressive actions easily result in messy parcel collisions and gridlock.

1.3 Kinematic Environment and System Modeling

The physical sorting system consists of two successive zones: a 5×5 AMS module with width $W = 1.0$ m and length $L = 1.0$ m (individual cell side $d_{AMS} = 0.2$ m), followed by a linear outfeed treadmill moving at a constant longitudinal speed $v_{treadmill}$.

Kinematic Equations of Motion

A purely kinematic model is used, neglecting inertial effects and parcel masses at low speed. A parcel i is driven by the actuator tile directly beneath its centroid (x_i, y_i) . Let (r_c, c_c) be the row and column of the active cell:

$$r_c = \min \left(5, \max \left(1, \left\lfloor \frac{y_i}{d_{AMS}} \right\rfloor + 1 \right) \right) \quad (1.1)$$

$$c_c = \min \left(5, \max \left(1, \left\lfloor \frac{x_i}{d_{AMS}} \right\rfloor + 1 \right) \right) \quad (1.2)$$

If the centroid lies on the AMS grid ($y_i \leq L$), its location is updated at each time step $\Delta t = 0.01$ s by the cell's physical speed $v_{AMS}(r_c, c_c)$ and steering angle $\theta_{AMS}(r_c, c_c)$:

$$x_i(t + \Delta t) = x_i(t) + v_{AMS}(r_c, c_c) \sin(\theta_{AMS}(r_c, c_c)) \Delta t \quad (1.3)$$

$$y_i(t + \Delta t) = y_i(t) + v_{AMS}(r_c, c_c) \cdot \cos(\theta_{AMS}(r_c, c_c)) \cdot \Delta t \quad (1.4)$$

When the centroid exits the grid ($y_i > L$) it is moved to the outfeed treadmill:

$$x_i(t + \Delta t) = x_i(t) \quad (1.5)$$

$$y_i(t + \Delta t) = y_i(t) + v_{treadmill} \Delta t \quad (1.6)$$

As it can be observed, the treadmill has no actuation capabilities and follows the equations of a linear motion with constant speed.

1.3.1 Collision Handling and Wall Boundaries

To prevent parcels from leaving the conveyor sideways the coordinates are clamped to the belt limits:

$$x_i(t) \in [w_i/2, W - w_i/2] \quad (1.7)$$

where w_i is the width of the parcel. Lateral and longitudinal overlaps between parcels on the AMS are resolved at each step through a rigid geometric push-out algorithm with

a contact tolerance $\epsilon_{contact} = 0.001$ m. If two active parcels i and j overlap, the positions are shifted symmetrically along the collision normal:

$$\mathbf{x}_i \leftarrow \mathbf{x}_i + \frac{\delta_{penetration}}{2} + \boldsymbol{\epsilon}_{contact} \quad \mathbf{x}_j \leftarrow \mathbf{x}_j - \frac{\delta_{penetration}}{2} - \boldsymbol{\epsilon}_{contact} \quad (1.8)$$

To mimic the operational assumption in the reference paper [2] that non-rigid packages slide past each other, the longitudinal front-to-back overlaps (y -axis overlaps) is allowed past the exit line.

1.4 Formulation as MDP

We model the singulation problem as a discrete-time discounted MDP defined by tuple $(\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma)$.

1.4.1 State Space $\mathcal{S}$

To cope with a variable number of parcels while keeping a fixed-size input layer for the neural networks, a grid-centric cell-indexed observation is used. If there are multiple centroids within one cell, the parcel closest to the exit line is mapped by the cell. The project compares three different observation vectors:

- **75-dimensional state (Square configuration):** maps 25 cells of the grid. Each cell $k \in [1, 25]$ has 3 variables:

$$\mathbf{s}_k = [x_i, y_i, d_i]^T, \quad (1.9)$$

where x_i and y_i are the centre-of-mass coordinates of the parcel and d_i its size.

- **108-dimensional state (Rectangular couples):** it combines the 25 grid cells (each storing 4 variables: x_i, y_i , width w_i and length l_i) plus the states of the first 2 exited parcels that are on the outfeed treadmill but have not crossed the evaluation line yet:

$$\mathbf{s} = [\mathbf{s}_{cell,1}, \dots, \mathbf{s}_{cell,25}, \mathbf{s}_{exit,1}, \mathbf{s}_{exit,2}]^T \in \mathbb{R}^{108} \quad (1.10)$$

- **112-dimensional state (Rectangular batches):** to allow for denser traffic, 3 parcels exit the 100 cell-based states:

$$\mathbf{s} = [\mathbf{s}_{cell,1} \quad \dots \quad \mathbf{s}_{cell,25} \quad \mathbf{s}_{exit,1} \quad \mathbf{s}_{exit,2} \quad \mathbf{s}_{exit,3}]^T \in \mathbb{R}^{112} \quad (1.11)$$

The third scenario is the case where up to three items can be generated at the same time.

1.4.2 Features and Dimensionality: Rationale

1. **The 75-Dimensional State (Scenario 1):** for square packages the width w_i and the length l_i are equal ($w_i = l_i = d_i$). This enables us to describe each cell with 3 metrics (x_i, y_i, d_i), which reduces the size of the network input layer by 25 compared to a 100-dimensional layout. This results in a reduced execution time for the forward and backward passes during PPO training.

2. **The 108 and 112-Dimensional States (Scenarios 2 and 3):** The introduction of rectangular parcels leads to a symmetry breaking which necessitates the independent tracking of the width w_i and the length l_i to permit the network to evaluate variations in aspect ratio (size range 5 cm to 45 cm). This gives a 100 dimensional representation ($25 \text{ cells} \times 4 \text{ variables}$).

Importantly, after a parcel’s centroid passes the AMS exit line ($y_{exit} = 1.0 \text{ m}$), it is no longer on the active grid, but moves on the outfeed treadmill towards the evaluation line ($y_{eval} = 2.0 \text{ m}$). This transit disables the actuation of the parcel but its position directly determines the target gap for incoming parcels. To avoid the agent being “blind” to parcels on the treadmill, the coordinates and dimensions of these exited parcels are added to the state vector:

- *Scenario 2 (Couples)*: the maximum number of simultaneously evaluated exited parcels is 2 resulting in an additional 8 states (108 in total).
- In *Scenario 3 (Batches)*, parcels are created in denser batches of 3, so up to 3 parcels can be on the treadmill at the same time, leading to 12 additional states (112 in total).

1.4.3 Action Space $\mathcal{A}$

The action space is $\mathcal{A} \subset \mathbb{R}^{50}$ with 50 continuous control variables. For each grid cell, the policy produces a normalized steering action $\bar{\theta} \in [-1, 1]$ and a normalized speed command $\bar{v} \in [-1, 1]$. Before application these are denormalized:

$$\theta_{AMS} = \theta_{lower} + (1 + \bar{\theta}) \frac{\theta_{upper} - \theta_{lower}}{2} \quad (1.12)$$

$$v_{AMS} = v_{lower} + (1 + \bar{v}) \frac{v_{upper} - v_{lower}}{2} \quad (1.13)$$

1.4.4 Reward Function $\mathcal{R}$

The reward signal consists of a dense shaping reward to guide learning during early iterations and a sparse event-based reward evaluated at the singulation KPI line ($y_{eval} = L + 1.0 \text{ m}$):

$$R_t = R_{shaping,t} + R_{event,t} - R_{action,t} \quad (1.14)$$

where:

- **Dense Gap Shaping ($R_{shaping,t}$)**: calculated at each simulation step to guide early policy learning, based on the longitudinal spacing of active parcels sorted by y :

$$R_{shaping,t} = \sum_{k=1}^{K-1} R_{gap}(p_k, p_{k+1}) \quad (1.15)$$

The shape of R_{gap} is tuned to the geometrical complexity of the configurations:

- **Square Models (Scenario 1):** A discrete step function is used:

$$R_{gap}(p_k, p_{k+1}) = \begin{cases} +0.1 & \text{if } \text{gap}_k > 0 \\ -2.0 & \text{if } \text{gap}_k < 0 \end{cases} \quad (1.16)$$

- **Rectangular Models (Scenarios 2 & 3):** A continuous bounded function is used to smooth gradients. A **Closing Velocity Penalty** is applied when the gap is less than 0.5 m:

$$R_{gap}(p_k, p_{k+1}) = \begin{cases} 0.5 \cdot \min\left(\frac{\text{gap}_k}{d_{\text{target}}}, 1.0\right) - R_{\text{closing}} & \text{if } \text{gap}_k > 0 \\ 0.5 \cdot \max\left(\frac{\text{gap}_k}{d_{\text{target}}}, -1.0\right) & \text{if } \text{gap}_k < 0 \end{cases} \quad (1.17)$$

where $d_{\text{target}} = 0.15$ m, and:

$$R_{\text{closing}} = 0.5 \cdot \max(0, v_{y,\text{trail}} - v_{y,\text{lead}}) \cdot \left(1 - \frac{\text{gap}_k}{0.5}\right) \quad (1.18)$$

In the above, $v_{y,\text{lead}}$ is the longitudinal velocity of the leading parcel (further downstream), and $v_{y,\text{trail}}$ is the velocity of the trailing parcel (further upstream).

- **Sparse Crossing Reward ($R_{event,t}$):** Computed on the evaluation line ($y_{eval} = 2.0$ m) when a parcel crosses it:

$$R_{event,t} = R_{crossing} + R_{sequence} \quad (1.19)$$

where the terms are defined as follows:

- **Crossing Score (All Scenarios):** Distance measure between crossing parcel and its predecessor:

$$R_{crossing} = \begin{cases} +100 & \text{if } \text{gap} \geq 0.15 \text{ m} \\ +25 & \text{if } 0 < \text{gap} < 0.15 \text{ m} \\ -50 & \text{if } \text{gap} \leq 0 \end{cases} \quad (1.20)$$

- **Sequence Enforcing (Rectangular Batches - Scenario 3):** Introduces an additional event penalty to deter out-of-order parcel arrivals:

$$R_{sequence} = \begin{cases} -200 & \text{if a parcel overtakes an earlier one} \\ 0 & \text{otherwise} \end{cases} \quad (1.21)$$

For all other cases (Scenarios 1 and 2), $R_{sequence} = 0$.

- **Action Regularization ($R_{action,t}$):** Penalizes control effort to promote smooth movements:

- **Square Model (Scenario 1):** Penalizes the magnitude of all 50 actions:

$$R_{action,t} = 0.001 \sum_{j=1}^{50} |a_j(t)| \quad (1.22)$$

- **Rectangular Models (Scenarios 2 & 3):** Focuses on specific steering angles θ_j and velocity deviations from the treadmill speed (1.9 m/s):

$$R_{action,t} = 0.001 \left(\sum_{j=1}^{25} |\theta_{AMS,j}| + \sum_{j=1}^{25} |v_{AMS,j} - 1.9| \right) \quad (1.23)$$

1.5 Hyperparameter choice

The hyperparameters of the PPO agent (see Table 1.1) have been selected to balance the physical timescales of the sorting module and the numerical demands of deep policy gradients.

Hyperparameter / Dimension	Square Couples	Rectangular Couples	Rectangular Batches
State space dimension ($ S $)	75	108	112
Action space dimension ($ A $)	50	50	50
Discount factor (γ)	0.99	0.99	0.99
GAE parameter (λ)	.95	.95	.95
Clip fraction (ϵ)	0.2	0.2	0.2
Actor/Critic learning rate	3×10^{-4}	3×10^{-4}	1.5×10^{-4}
Entropy coefficient	0.01	0.01	0.002
Roll-out horizon (steps)	2048	2048	2048
Mini-batch size	256	256	256
Epochs per update	10	10	10
Optimizer	Adam	Adam	Adam
Maximum training episodes	6000	6000	6000
Max steps per episode	1800	1800	1800

Table 1.1: Hyperparameters for PPO Agent and Environment Dimensions for Different Scenarios.

1.5.1 Discount Factor $\gamma = 0.99$

The simulator is run with a time step of $\Delta t = 0.01$ s. The effective planning horizon with the discount factor $\gamma = 0.99$ is:

$$H_{eff} = \frac{1}{1 - \gamma} = 100 \text{ timesteps} = 1.0 \text{ s} \quad (1.24)$$

This is a horizon grounded in physics. The active grid is 1.0 m long and the nominal transit speed is between 0.5 m/s and 2.2 m/s, so a parcel is on the active grid for roughly 0.5 s to 1.0 s. Setting $H_{eff} = 1.0$ s, we make sure that the critic network takes into account the total effects of cell actions until the exit, but not too much variance from distant, unrelated future episodes.

1.5.2 GAE $\lambda = 0.95$

We estimate the policy advantage function using Generalized Advantage Estimation (GAE). The value $\lambda = 0.95$ is a compromise between the high bias of Temporal Difference (TD)(0) advantage ($\lambda = 0$) and the high variance of Monte Carlo returns ($\lambda = 1$).

In our setting, parcel collisions cause sharp discontinuities in the reward, and a slightly smaller λ stabilizes the advantage estimation against local transition noise.

1.5.3 Weight of the Entropy Loss

In the continuous action space ($\theta_{AMS} \in [-\pi/4, \pi/4]$, $v_{AMS} \in [0.5, 2.2]$ m/s) policies are represented as multivariate Gaussian distributions. In the absence of entropy regularization, the standard deviation, σ , of the policy can collapse too soon, leading to a deterministic policy before exploring the full action space. The 0.01 entropy weight helps to keep the minimum exploration push, which helps the steering angles to explore the alternative trajectories around the obstacles.

1.5.4 Experience Horizon (2048) and Mini-Batch Size (256)

The agent collects 2048 steps of experience from parallel workers before policy updates each iteration. This gives a statistically representative sample of parcel trajectories. In the update step, we split the 2048 transitions into 8 mini-batches of size 256. This mini-batch size is large enough to average out the high gradient variance caused by sudden collision events, yet small enough to allow efficient GPU matrix execution.

1.6 Test Scenarios

The experimental campaign is organized to systematically assess the generalization capabilities and robustness of the controller under different stress levels following three operational configurations of increasing physical and dimensional complexity matching the subdirectories of the project:

1. **Square Couples (Scenario 1):** This configuration is the baseline task. It uses square packages ($w_i = l_i = d_i$) produced in pairs (2 packages per batch, 10 packages total per episode) with a constant interval of 0.75 s as implemented in `RL_environment_two_square_packages.m`. The geometric symmetry keeps the state space compact at 75 dimensions (3 metrics per grid cell: centroid x , y and dimension d).
2. **Rectangular Couples (Scenario 2):** This scenario considers aspect ratio variations by employing rectangular parcels with independent random variables of width w_i and length l_i . The state representation is augmented to 108 dimensions (with separate width and length channels for the 25 cells, and tracking up to 2 exited parcels). The traffic density is kept at 2 parcels per batch, so as to isolate the effect of geometry on the policy.
3. **Rectangular Batches (Scenario 3):** To test the agent’s capability to deal with highly congested streams under random traffic pressure, rectangular parcels are generated in variable batch sizes of 1, 2 or 3 items spawned simultaneously, for a total of 10 parcels per episode. To balance the traffic density on the grid, the simulation implements a *dynamic generation interval* based on the batch size:

- A one-parcel spawn ($N = 1$) induces a post-generation delay of 0.6 s;
- Two-parcel spawn ($N = 2$) causes delay of 0.9 s;
- A three parcel spawn ($N = 3$) leads to a delay of 1.4 s.

The state space is augmented to 112 dimensions, comprising the 100-dimensional grid observation ($25 \text{ cells} \times 4 \text{ variables}$) plus 12 states to track up to 3 exited parcels on the outfeed treadmill.

1.6.1 Training Procedure

The three scenarios have different state-space dimensionalities (75, 108 and 112 dimensions respectively), so policies are trained independently from scratch and weight transfer is not used. Each agent is trained with a variable training budget (6000 episodes for Square Couples, 6000 episodes for Rectangular Couples, and 6000 episodes for Rectangular Batches to accommodate the complex multi-body collision dynamics in dense traffic). The weights are initialized using random Glorot initialization to ensure stable initial gradient updates.

1.7 Results

1.7.1 Learning Curve in a Sample Scenario and Models Performances

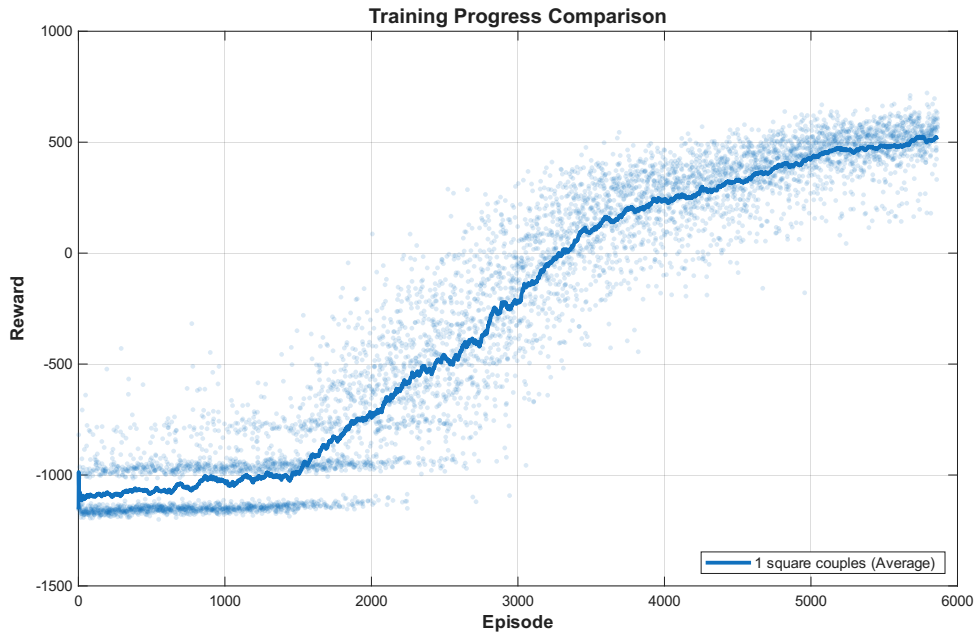


Figure 1.1: Training progress of the PPO agent on the *Square Couples* configuration.

To evaluate the sample efficiency and stability of the training process, the reward trajectory of the baseline agent on the *Square Couples* task over approximately 6,000 episodes

is examined. Figure 1.1 shows the training curve, where the solid blue line represents the running average of the episode return, while the scattered light-blue points denote the raw returns obtained in individual episodes. The training process exhibits a classic three-phase reinforcement learning convergence profile:

- **Exploration Phase (Episodes 0 to 1,500):** In the initial stage, the policy behaves close to random exploration. The average reward remains low and relatively flat, hovering around $-1,100$ and $-1,000$. During this phase, the agent frequently incurs negative rewards due to parcel collisions, lateral belt boundary crossings, and the inability to establish positive exit gaps. The high dispersion of the scattered points reflects the high variance of random exploration.
- **Rapid Learning Phase (Episodes 1,500 to 3,500):** Once the agent discovers the basic steering heuristics that prevent immediate collisions, the reward curve experiences a sharp, monotonic increase, climbing from $-1,000$ to approximately $+200$. This steep gradient demonstrates the efficiency of the PPO clipped surrogate objective, which allows for stable policy updates without destabilizing the network weights even in the presence of discontinuous collision dynamics.
- **Policy Refinement Phase (Episodes 3,500 to 6,000):** Beyond episode 3,500, the learning curve transitions into an asymptotic refinement stage. The reward growth slows down but increases steadily towards $+500$. In this regime, the agent focuses on fine-tuning the continuous actuator controls (steering angles and velocities) to maximize the sparse crossing bonuses at the evaluation line. The cloud of episode returns shifts upward and narrows, indicating that the agent has converged to a highly stable, deterministic policy that is robust to the random coordinates and dimensions of incoming parcel batches.

Additionally, the mean policies of the three agents were tested over 1000 independent validation episodes under their respective operational scenarios. The evaluation metrics focus on the *Singulation Success Rate (%)*, which measures the percentage of consecutive parcel pairs exiting the module with a strictly positive longitudinal gap ($gap > 0$).

Figure 1.2 illustrates the distribution of the singulation accuracy across the three configurations, combining a box plot with a swarm chart overlay where each marker represents a single test episode.

The empirical results reveal a clear relationship between the geometric and congestion complexity of the scenario and the stability of the learned policy:

- **Square Couples (Scenario 1):** The baseline configuration achieves an outstanding mean singulation accuracy of **99.9%**. The distribution is highly concentrated, with almost all episodes achieving a perfect 100% success rate. Due to the geometric symmetry of square parcels ($w_i = l_i = d_i$) and the controlled low-density generation (fixed batches of 2), the state transition dynamics are highly predictable, allowing the agent to easily coordinate the sorting actions.
- **Rectangular Couples (Scenario 2):** Introducing aspect ratio variations ($w_i \neq l_i$) slightly increases the task difficulty, yielding a mean accuracy of **99.3%**. While the median performance remains at 100%, the variance increases slightly, with a few

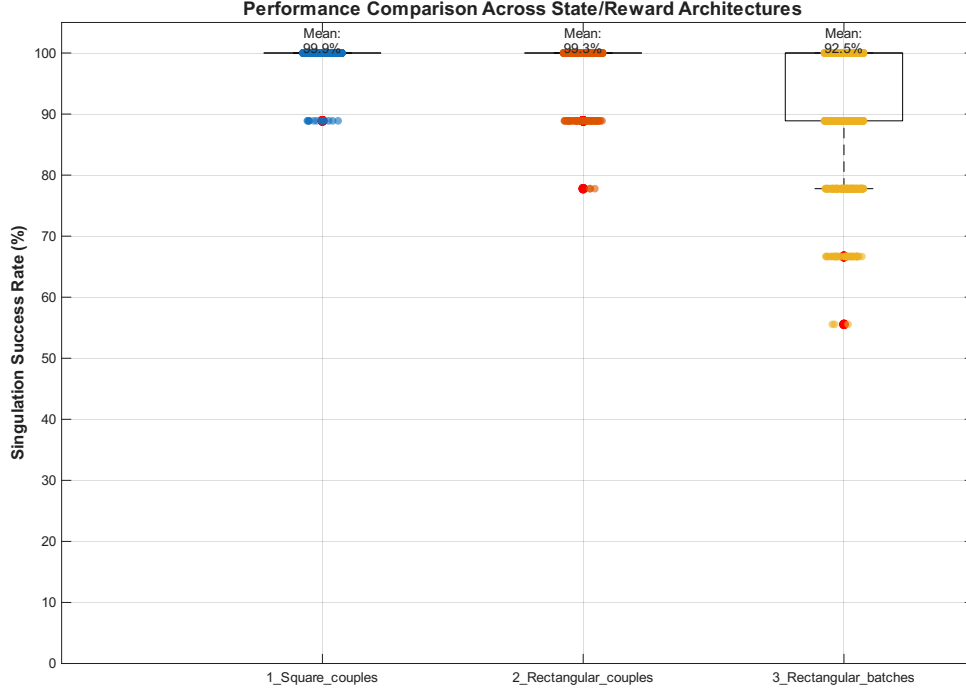


Figure 1.2: Performance comparison of the trained PPO controllers across the three operational configurations. Markers represent individual evaluation episodes, with red dots denoting statistical outliers.

outlier episodes dropping to 89% and 78%. This minor degradation is caused by unfavorable initial orientations and dimensions of rectangular parcels, which require more precise steering adjustments to ensure the target gap.

- **Rectangular Batches (Scenario 3):** This represents the most challenging and realistic setup, resulting in a mean singulation accuracy of **92.5%**. The performance distribution displays a significantly wider spread, with the interquartile range spanning from 89% to 100% and worst-case outliers dropping to 55%. The increased variance and lower mean are directly attributed to:

1. *Multi-body contact dynamics:* The simultaneous spawning of up to 3 rectangular parcels increases the frequency of physical collisions on the grid, which are resolved symmetrically along the contact normal, perturbing the agent’s planned trajectories.
2. *Sequence preservation constraints:* The strict enforcement of the FIFO (First-In, First-Out) order via the overtaking penalty ($R_{sequence} = -200$) forces the agent to prioritize sequence alignment over pure throughput, occasionally leading to local grid-jams and tighter exit gaps in highly congested episodes.

1.7.2 Robustness and Zero-Shot Generalization under Constant Batch Sizes

Finally, to evaluate the generalization capabilities and robustness of the policy, the agent trained on the *Rectangular Batches* scenario, where spawn sizes are randomized between

1 and 3 parcels, is stress-tested under three fixed-batch regimes of increasing congestion: constant batches of 2, 3, and 4 parcels. The latter case (4 parcels) represents a strictly out-of-distribution (OOD) task, as the agent never experienced spawns larger than 3 during training.

Figure 1.3 reports the distribution of the Singulation Success Rate (%) across 1000 evaluation episodes for the three test cases.

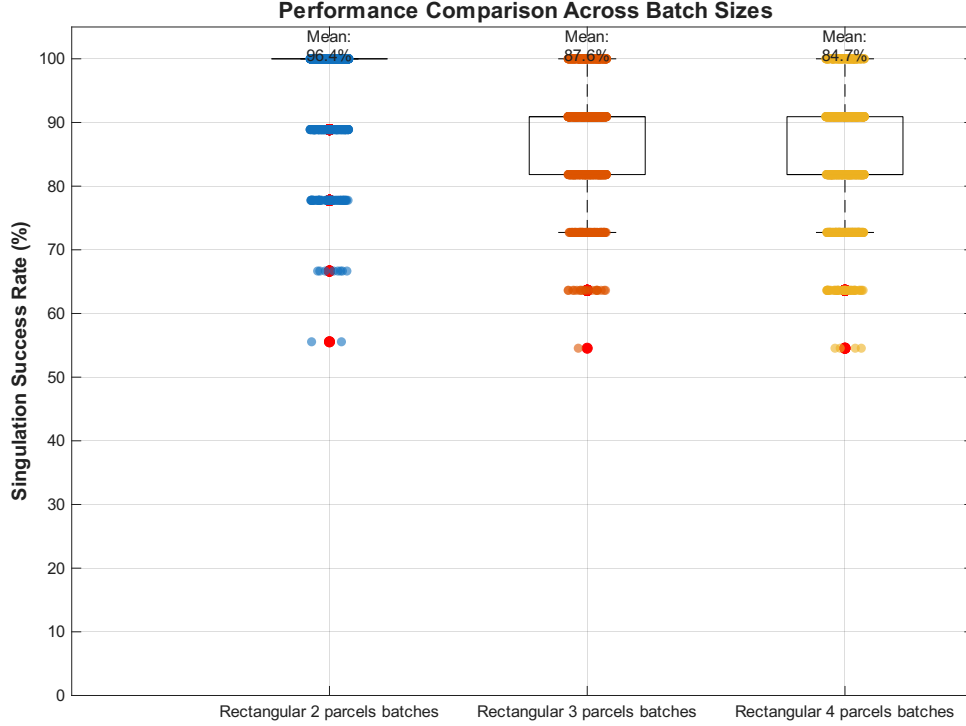


Figure 1.3: Zero-shot performance comparison of the Rectangular Batches agent evaluated under constant spawn sizes of 2, 3, and 4 parcels.

The comparative analysis reveals how the agent adapts to constant traffic pressure and generalizes to unseen densities:

- **Constant 2-Parcel Batches:** The agent achieves a high mean singulation accuracy of **96.4%**. Since the policy was trained to handle up to 3 simultaneous parcels, a constant spawn size of 2 represents a slightly less congested state. The agent manages the grid flow with minimal variance, and the distribution is tightly clustered around 100%.
- **Constant 3-Parcel Batches:** The mean performance decreases to **87.6%**. During training, 3-parcel spawns were randomized, meaning the average traffic density was lower. Forcing exactly 3 parcels at every single spawn step subjects the controller to sustained maximum nominal traffic pressure. The continuous presence of multiple bodies on the grid increases the frequency of physical overlaps and sequence-preservation conflicts, widening the interquartile range (spanning from 82% to 91%) and increasing the occurrence of outlier episodes.

- **Constant 4-Parcel Batches (Out-of-Distribution):** When subjected to a completely unseen regime of 4 simultaneous parcels, the agent maintains a remarkably high mean singulation accuracy of **84.7%**. This robust performance is mathematically and physically justified by two factors:
 1. **Spawn Interval Saturation:** In the environmental code, the generation delay is saturated for any batch size larger than or equal to 3 items, imposing a constant interval of 1.4 s (`if genCount >= 3, interval = 1.40s`). Consequently, the 4-parcel batch does not overload the grid timing-wise, as it inherits the same 1.4 s clearing window designed for 3-parcel batches.
 2. **Geometric Clamping:** Since the total batch volume is constrained, spawning 4 parcels forces the generator to produce smaller average physical sizes for each package. These smaller geometries fit more easily within the conveyor boundaries, allowing the policy to coordinate lateral separations without causing catastrophic grid jams.

The policy exhibits a graceful degradation under extreme OOD congestion rather than a catastrophic failure, proving the structural generalization of the learned collision-avoidance rules.

Chapter 2

Python project

2.1 Dynamic Modeling of the Compliant Base Robot System

The second project deals with the dynamic modelling and control of a Cartesian impedance-controlled manipulator mounted on a compliant non-rigid base. Physically, this system is schematically represented as a two-mass spring-damper system connected in series. To derive the coupled equations of motion in free-space (i.e. ignoring environment contact terms, $K_e = 0$ and $D_e = 0$), the reference is the modeling framework proposed by Roveda et al. [1]. We define the generalized coordinates of the system:

- $x_b(t) \in \mathbb{R}$: absolute displacement of the compliant robot base (represented by the red mass M_b which acts as a passive, non-directly controllable Degree of Freedom (DoF));
- $x_r(t) \in \mathbb{R}$: relative position of the robot end-effector (represented by the purple mass M_r) relative to the compliant base;
- $y(t) = x_b(t) + x_r(t) \in \mathbb{R}$: absolute position of the end-effector in the world frame.

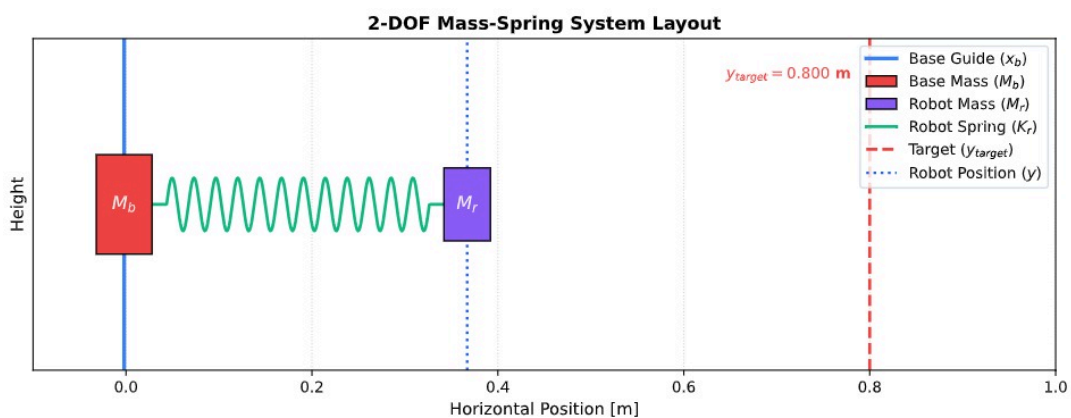


Figure 2.1: 2-DOF Mass-Spring System Layout.

The compliant base is connected to a fixed ground support ($x_b^0 = 0$) by an elastic support with mass M_b , damping D_b , and stiffness K_b . The robot is controlled through a Cartesian impedance controller, which enforces a desired dynamic behavior on the end-effector. This controller is represented by a virtual spring-damper system with stiffness K_r and damping D_r in relation to a commanded set-point trajectory $x_r^0(t)$ (representing the control input $u(t)$).

In the Lagrangian approach, the potential energy V , kinetic energy T and Rayleigh dissipation function D of the coupled system in the free space are expressed as:

$$V = \frac{1}{2}K_r(x_r - x_r^0)^2 + \frac{1}{2}K_b x_b^2 \quad (2.1)$$

$$T = \frac{1}{2}M_r(\dot{x}_b + \dot{x}_r)^2 + \frac{1}{2}M_b \dot{x}_b^2 \quad (2.2)$$

$$D = \frac{1}{2}D_r \dot{x}_r^2 + \frac{1}{2}D_b \dot{x}_b^2 \quad (2.3)$$

The coupled linear dynamic equations of the system are derived by applying the Euler-Lagrange's equations:

$$\begin{cases} (M_r + M_b)\ddot{x}_b + M_r\ddot{x}_r + D_b\dot{x}_b + K_b x_b = 0 \\ M_r(\ddot{x}_r + \ddot{x}_b) + D_r\dot{x}_r + K_r(x_r - x_r^0) = 0 \end{cases} \quad (2.4)$$

Rearranging the second equation of (2.4) isolates the force interaction $f_{int} = -M_r(\ddot{x}_r + \ddot{x}_b)$ exerted by the impedance-controlled manipulator to its compliant base:

$$f_{int} = D_r\dot{x}_r + K_r(x_r - x_r^0) \quad (2.5)$$

then allows us to rewrite the passive base dynamics as a classical second order mass-spring-damper system excited by the internal robot forces:

$$M_b\ddot{x}_b + D_b\dot{x}_b + K_b x_b = f_{int} \quad (2.6)$$

2.1.1 The Underactuated Reachability Challenge

The main objective of the project is to control the active relative position $x_r(t)$ by the set-point command $x_r^0(t)$ to bring the absolute end-effector position $y(t)$ to a target coordinate y_{target} . Critically, the physical operative limits of the manipulator restrict the relative extension of the manipulator, bounding the active DoF:

$$x_r(t) \in [-x_{limit}, x_{limit}] \quad (2.7)$$

Because the target position is beyond the nominal reach of the robot when the base is at rest (i.e., $y_{target} > x_{limit}$), a static extension of the arm is not enough to complete the task. To overcome this underactuation constraint, the controller has to dynamically excite the passive DoF. The system is driven to the natural resonant frequency of the compliant base by commanding an oscillatory reference trajectory such as a chirp modulated by feedback control strategies like Velocity Feedback, MPC or HDP. This resonance leads to large oscillations in $x_b(t)$. The maximum extension of the manipulator occurs when the base displacement is positive ($x_b > 0$) and the manipulator extension is at the maximum ($x_r = x_{limit}$).

2.2 State Estimation, Control and Parameters Optimization

In this section, the design, training and implementation of a data-driven state estimator and the active resonance controllers designed to tackle the underactuated reachability challenge are presented. Moreover, a multi-resolution parameter optimization algorithm is proposed to search the optimal virtual stiffness, damping and energy-pumping gains of the system.

2.2.1 LSTM versus FNN for State Estimation

The absolute position of the end-effector in the world frame is given by $y(t) = x_b(t) + x_r(t)$. The robot joint encoders directly measure the relative position of the manipulator $x_r(t)$, but the absolute displacement $x_b(t)$ and the velocity $v_b(t)$ of the compliant base are not directly observable because of industrial and practical constraints (for example, no external tracking camera or linear encoder on the base). A data-driven *virtual sensor* is developed to reconstruct the state of the passive DoF.

Theory of Feedforward Neural Networks (FNNs)

A normal FNN directly maps an input vector x to an output vector y through a series of fixed feed-forward layers. The hidden state vector $h^{[l]}$ for layer l is defined as:

$$h^{[l]} = \sigma(W^{[l]} \cdot h^{[l-1]} + b^{[l]}) \quad (2.8)$$

where $W^{[l]}$ is the weight matrix, $b^{[l]}$ is the bias vector and $\sigma(\cdot)$ is a non-linear activation function. FNNs are static mapping operators. They assume that the input sample at time step t is independent from the input sample at time step $t - 1$.

Static Architectures and Their Physical Limits in Inertial Dynamics

In a second order mechanical system the state at time t is intrinsically coupled to its history. The dynamics of the passive base are described by:

$$M_b \ddot{x}_b(t) + D_b \dot{x}_b(t) + K_b x_b(t) = f_{int}(t) \quad (2.9)$$

For a static neural network to estimate the instantaneous base position $x_b(t)$ and velocity $v_b(t)$, the input vector would need sufficient temporal context. For a given instantaneous relative robot position $x_r(t)$ and actuator command $x_{r0}(t)$, the physical base can be in diametrically opposite dynamic states (e.g. maximum compression vs. maximum extension) depending on whether the system is accelerating, decelerating or reversing phase. A single static snapshot FNN is not able to reconstruct the temporal derivatives or the inertial memory of the system and hence fails.

2.2.2 LSTM Model

The Long Short-Term Memory (LSTM) [3] net architecture is used to account for the time-varying, history-dependent nature of the dynamics. The LSTM solves the vanishing

and exploding gradient problems of traditional Recurrent Neural Networks (RNN)s by introducing a specific cell state c_t and three gating mechanisms that control the information flow. The gates and state updates of the LSTM cell at each time step t are computed given the previous hidden state h_{t-1} and the current input vector x_t as:

1. **Forget Gate (f_t):** Controls how much of history to throw away:

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \quad (2.10)$$

2. **Forget Gate (f_t):** Determines what new information from the current input adds to the cell state:

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \quad (2.11)$$

3. **Candidate Cell State ($\tilde{c}_t$):** Creates a vector of new candidate values to insert into memory:

$$\tilde{c}_t = \tanh(W_c \cdot [h_{t-1}, x_t] + b_c) \quad (2.12)$$

4. **Cell State Update (c_t):** Update the persistent memory with the gated past state and gated candidate state:

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t \quad (2.13)$$

where $\odot$ is the Hadamard (element-wise) product. Because the update rule for c_t is linear and additive, gradients can flow backwards in time without decaying exponentially.

5. **Output Gate (o_t):** Controls the information to output as the hidden state:

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o) \quad (2.14)$$

6. **Updating Hidden State (h_t):** Computes the output of the recurrent layer for the current step:

$$h_t = o_t \odot \tanh(c_t) \quad (2.15)$$

2.3 Dataset Generation and Deep Learning Training Pipeline

A reliable training pipeline is developed in Python to build the virtual sensor, using synthetic physics simulations to generate training data.

2.3.1 Input and Output Vector Formulations

The LSTM network is trained to receive three physical input variables, measured on the manipulator, and to return the estimated state of the passive base. The input vector is $X(t) \in \mathbb{R}^3$ and the target output vector is $Y(t) \in \mathbb{R}^2$:

$$X(t) = [x_r(t) \quad v_r(t) \quad x_{r0}(t)]^T, \quad Y(t) = [x_b(t) \quad v_b(t)]^T \quad (2.16)$$

where the relative velocity of the end-effector is given by $v_r(t) = \dot{x}_r(t)$.

2.3.2 Synthetic dataset generation by means of Chirp excitation

To ensure generalization over a wide range of frequencies and amplitudes, a synthetic dataset consisting of 1200 independent physical simulations is generated. Each simulation was run for 20 s with an integration time-step $\Delta t = 0.01$ s, which leads to time-series sequences of 2000 steps. The system is excited in each simulation with a chirp signal $x_{chirp}(t)$ with randomized parameters from uniform distributions to cover the dynamic range of the physical hardware:

$$\text{Chirp Amplitude : } A \sim \mathcal{U}(0.03, 0.08) \text{ m} \quad (2.17)$$

$$\text{Initial Frequency : } f_0 \sim \mathcal{U}(0.05, 0.2) \text{ Hz} \quad (2.18)$$

$$\text{Final Frequency : } f_1 \sim \mathcal{U}(4.5, 6.5) \text{ Hz} \quad (2.19)$$

2.3.3 Data Normalization

The physical variables are on different scales (position in centimeters, velocity in meters per second), so the training is stabilized by normalizing the data using a standard Z-score normalization:

$$x_{norm} = \frac{x - \mu_x}{\sigma_x} \quad (2.20)$$

The normalization parameters (μ and σ) are calculated separately for the input and output variables. They are exported to JSON configurations (`scalers_simple.json` and `scalers_complex.json`) to ensure consistency during real-time client-side inference.

2.3.4 Situations of Custom Training

Since the project requires a high accuracy of estimation, the following model is developed. The design is intended for non-linear, resonant regimes with unilateral rigid impacts at $x_r = 0$ m. Nominal robot parameters are changed to favor elongation: $M_r = 10.0$ kg, $K_r = 600$ N/m, and $h_r = 0.15$. The network architecture is a stack of two LSTM layers with 64 hidden units each because of severe non-linearities and inelastic shocks. The model is trained in PyTorch with Mean Squared Error (MSE) loss function:

$$\mathcal{L}_{MSE} = \frac{1}{N} \sum_{i=1}^N (Y_i - \hat{Y}_i)^2 \quad (2.21)$$

The model is trained using the Adam optimizer with an initial learning rate of 0.003. Training is run for 120 epochs.

2.4 Active Resonance Control

Three different control laws are proposed to inject energy into the passive DoF so that it can reach the target coordinate y_{target} which is beyond the static reach of the robot.

2.4.1 Classical velocity feedback (resonance pumping)

The velocity feedback controller is designed to provide virtual negative damping to drive the passive base into resonance. The control input is given by:

$$x_{r0}(t) = x_{chirp}(t) - G_{elg} \cdot \hat{v}_b(t) \quad (2.22)$$

where $\hat{v}_b(t)$ is the estimated base velocity by the LSTM virtual sensor and G_{elg} is the elongation gain.

Energy injection proof for $G_{elg} > \frac{D_b}{K_r}$

As an example of the physical mechanism of virtual negative damping, consider the Cartesian force $F_{robot}(t)$ generated by the impedance-controlled manipulator, where $x_r(t)$ is the relative coordinate of the robot end-effector in relation to the compliant base and the force exchanged between the robot and the base is:

$$F_{robot}(t) = K_r(x_r(t) - x_{r0}(t)) + D_r\dot{x}_r(t) \quad (2.23)$$

Replacing the control command $x_{r0}(t)$ in the force equation, we get:

$$F_{robot}(t) = K_r(x_r(t) - x_{chirp}(t) + G_{elg}\hat{v}_b(t)) + D_r\dot{x}_r(t) \quad (2.24)$$

Assuming perfect state estimation (i.e. $\hat{v}_b(t) \approx \dot{x}_b(t)$), the force expression reduces to:

$$F_{robot}(t) \approx K_r(x_r - x_{chirp}) + K_r G_{elg} \dot{x}_b(t) + D_r \dot{x}_r(t) \quad (2.25)$$

Neglecting the non-linear impact term $F_{NL}(x_b)$ for simplicity, the differential equation governing the passive base is:

$$M_b\ddot{x}_b(t) + D_b\dot{x}_b(t) + K_b x_b(t) = F_{robot}(t) \quad (2.26)$$

Substituting the expression for $F_{robot}(t)$ and bringing all terms dependent on the base velocity $\dot{x}_b(t)$ to the left-hand side, we get:

$$M_b\ddot{x}_b(t) + \underbrace{(D_b - K_r G_{elg})}_{D_{total}} \dot{x}_b(t) + K_b x_b(t) = K_r (x_r(t) - x_{chirp}(t)) + D_r \dot{x}_r(t) \quad (2.27)$$

The total equivalent damping coefficient perceived by the passive base is given by:

$$D_{total} = D_b - K_r G_{elg} \quad (2.28)$$

Note that the elongation gain G_{elg} scales the position set-point command of the virtual spring, which scales with the stiffness K_r and not the robot damping D_r .

Analysis of the impact of G_{elg} on the stability of the system:

- **$G_{elg} = 0$ (no feedback):** Total damping is positive ($D_{total} = D_b$). The natural structural dissipation reduces the base oscillations.
- **Partial Dissipation ($G_{elg} < \frac{D_b}{K_r}$):** The total effective damping is decreased but still positive ($D_{total} > 0$). This limits the growth of the oscillations.

- $G_{elg} = \frac{D_b}{K_r}$ (**No Friction / Transparency**): The feedback force exactly cancels the physical base damping ($D_{total} = 0$)
- **Active Resonance Pumping** ($G_{elg} > \frac{D_b}{K_r}$): The total equivalent damping coefficient is less than zero ($D_{total} < 0$). The closed-loop system becomes unstable and the base oscillations grow exponentially and so the absolute position $y(t)$ can reach y_{target} . The physical parameters of the experimental setup are $M_b = 70$ kg, $K_b = 4000$ N/m, $h_b = 0.25$, with the optimal robot stiffness $K_r = 400$ N/m. The natural base damping is $D_b = 2\sqrt{K_b M_b} h_b \approx 264.5$ Ns/m. The critical instability threshold is given by:

$$G_{elg,crit} = \frac{D_b}{K_r} = \frac{264.5}{400} \approx 0.66 \quad (2.29)$$

This threshold explains why a gain of $G_{elg} = 0.50$ is not sufficient to overcome structural damping while a gain of $G_{elg} = 1.20 > 0.66$ is successful in pumping energy, inducing the base resonance required to reach the target.

2.4.2 MPC Heuristic

The physical implementation of the controller is characterized by time delays from sensor processing, LSTM inference latency, and actuator response times, which sum up to $\tau \approx 50$ ms. Due to the sensitivity of resonance pumping to phase synchronization, phase lag can make the robot absorb energy, rather than inject it, damping the system or causing instability. To compensate for this latency a heuristic MPC strategy is implemented. The controller calculates the estimated base acceleration $\hat{a}_b(t)$ using backward differences:

$$\hat{a}_b(t) = \frac{\hat{v}_b(t) - \hat{v}_b(t - \Delta t)}{\Delta t} \quad (2.30)$$

The controller predicts the base velocity $\hat{v}_{b,pred}$ at the look-ahead horizon $\tau = 0.05$ s based on a Taylor expansion.

$$\hat{v}_{b,pred}(t + \tau) = \hat{v}_b(t) + \tau \cdot \hat{a}_b(t) \quad (2.31)$$

The predictive control law is then obtained as:

$$x_{r0}(t) = x_{chirp}(t) - G_{elg} \hat{v}_{b,pred}(t + \tau) \quad (2.32)$$

The physical actuation is in phase with the base velocity by sending the command in phase-advance relative to the current measured state, to ensure optimal energy transfer.

2.4.3 HDP Policy

To prevent the system from divergent oscillations or violent impacts once the target is reached, a dual-mode hybrid policy is implemented. The controller computes the instantaneous distance to the target as:

$$d_{target}(t) = y_{target} - (x_b(t) + x_r(t)) \quad (2.33)$$

The policy has two modes based on a spatial safety threshold of 1.5 cm:

- **Case A: Energy Accumulation** ($d_{target}(t) > 0.015 \text{ m}$): The robot is way off target. The controller operates in pure velocity feedback mode to maximize the energy injection:

$$x_{r0}(t) = x_{chirp}(t) - G_{elg} \cdot \hat{v}_b(t) \quad (2.34)$$

- **Case B: Active Damping** ($d_{target}(t) \leq 0.015 \text{ m}$): The robot is moving towards the target. To reduce overshoot and prevent damage to the structure the controller uses active relative damping based on the physical velocity of the robot $v_r(t)$ and a damping gain $G_{damping} = 0.6$:

$$x_{r0}(t) = x_{chirp}(t) - G_{elg} \cdot \hat{v}_b(t) - G_{damping} \cdot v_r(t) \quad (2.35)$$

2.5 Multi-Resolution Optimization

Then, a multi-resolution optimization algorithm is designed to search for the optimal parameter set to tune the controller to the natural resonance frequency of the coupled system.

2.5.1 Formulation of Optimization Space

The optimization parameter vector is $\theta = [K_r, h_r, G_{elg}]^T$. The coupled system has seven parameters; the remaining variables are frozen. The base parameters (M_b, K_b, h_b) are structural boundaries which cannot be modified by the software. The robot mass M_r is physically limited to 10.0 kg and the chirp parameters are the exploration signal. The optimization is therefore restricted to the three main software parameters.

The domain of the search space $\mathcal{D}$ is then:

$$\mathcal{D} = \left\{ \theta \in \mathbb{R}^3 : \begin{array}{l} 200 \leq K_r \leq 1500 \text{ N/m} \\ 0.05 \leq h_r \leq 0.25 \\ 0.0 \leq G_{elg} \leq 1.50 \end{array} \right\} \quad (2.36)$$

The Cartesian damping coefficient is updated dynamically as $D_r = 2\sqrt{K_r M_r} h_r$.

2.5.2 Stability-Aware Objective Function

The objective function $\mathcal{J}(\theta)$ is formulated to maximize the peak absolute position of the end-effector. A stability constraint is imposed to prevent the algorithm from choosing divergent parameter combinations. The energy variation ΔE on two evaluation windows at the end of the simulation, $W_1 = [10 \text{ s}, 15 \text{ s}]$ and $W_2 = [15 \text{ s}, 20 \text{ s}]$, is evaluated:

$$\mathcal{J}(\theta) = \begin{cases} \max_t (x_b(t) + x_r(t)) & \text{if } \Delta E < \text{Threshold} \\ -1000 & \text{if unstable (divergent)} \end{cases} \quad (2.37)$$

2.5.3 The 4-stage optimization algorithm

The optimal parameters are searched using a four-stage process to achieve high accuracy:

Stage 1: Global Search (Multi-start with Jittering)

The first step is to define a discrete search grid Θ_{base} over the domain $\mathcal{D}$. Restarts are performed to prevent spatial aliasing. $N = 4$ restarts are used. For the initial restart $r = 0$, the exact grid points are evaluated. For restarts $r \in \{1, 2, 3\}$, random perturbations followed by mathematical clamping are applied:

$$\tilde{\theta}_i^{(r)} = \text{Clamp}(\theta_i + \mathcal{U}(-H_\theta, +H_\theta), \theta_{min}, \theta_{max}) \quad (2.38)$$

where H_θ is the noise bound (e.g., 50 N/m for K_r , 0.02 for h_r).

Stage 2: LSTM Validation (Surrogate Check)

In this stage, we use an ‘‘Oracle’’ model (a fourth-order Runge-Kutta numerical integration assuming perfect state feedback $\hat{v}_b = v_b$) to quickly find promising areas. The best candidate parameters θ_{oracle}^* from Stage 1 are validated by replacing the perfect feedback velocity v_b with the actual LSTM prediction $\hat{v}_b$. The actual score is obtained by running a closed-loop simulation accounting for the network estimation errors and latencies:

$$\mathcal{J}_{real}(\theta) = \text{Score}_{\text{LSTM}}(\theta) \quad (2.39)$$

This step establishes the globally optimal stable parameter set: $\theta_{best}^{(0)}$.

Stage 3: Local Grid Refinement

The local search neighborhood around $\theta_{best}^{(0)}$ is evaluated with a scaling radius $R = 0.15$ (15%). We sample $N_{fine} = 7$ points evenly spaced along each parameter dimension. The local boundaries are computed as:

$$\text{Low} = \max(\theta_{min}, \theta_{best}^{(0)}(1 - R)) \quad (2.40)$$

$$\text{High} = \min(\theta_{max}, \theta_{best}^{(0)}(1 + R)) \quad (2.41)$$

and we fill the fine search grid with points:

$$\theta_{fine}(k) = \text{Low} + \frac{k}{N_{fine} - 1}(\text{High} - \text{Low}) \quad k \in [0, 6] \quad (2.42)$$

Evaluating the combinations on the local grid ($7^3 = 343$ points) gives the refined parameters $\theta_{best}^{(1)}$. Furthermore, to keep the computational latency low in the client browser, the real LSTM validation is performed only on the top 10 candidate parameter sets pre-filtered by the Oracle model.

Stage 4: Micro-Tuning

The local refinement procedure is repeated in the vicinity of $\theta_{best}^{(1)}$ shrinking the search neighborhood:

- **Decreased Radius:** $R = 0.05$ (5%).
- **Grid Density:** $N_{micro} = 5$ ($5^3 = 125$ points for the evaluation).

Similarly, the last LSTM-based validation is limited to the top 5 candidates of the micro-grid to output the best parameters θ_{opt} . Finally, the impedance controller is carefully adjusted to the resonant frequency of the system.

2.6 Results

In this section, the performance of the proposed state estimation and control frameworks and the convergence properties of the multi-resolution optimization algorithm are reported.

LSTM Virtual Sensor Performance

To validate the accuracy of the data-driven state estimator under passive, uncontrolled base oscillations, the estimated base displacement $\hat{x}_b(t)$ is compared with the true base displacement $x_b(t)$ obtained from the physical simulator. Figure 2.2 presents the time response of the estimator and the instantaneous error in the tracking $e(t) = x_b(t) - \hat{x}_b(t)$.

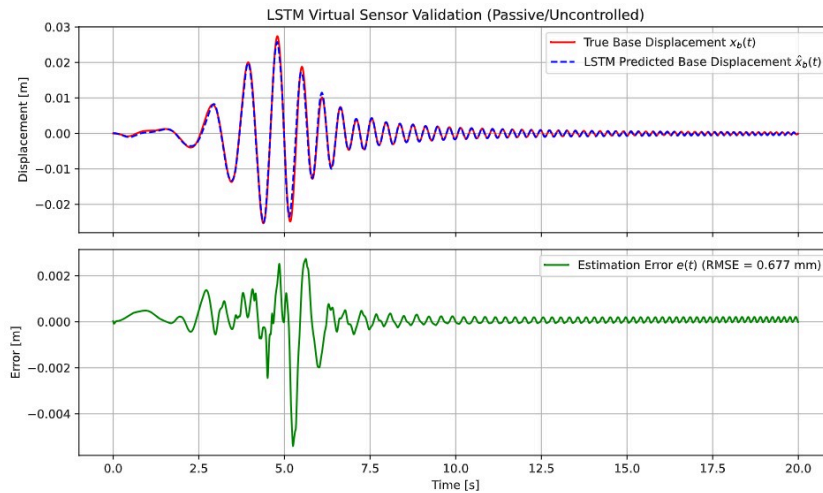


Figure 2.2: LSTM Virtual Sensor validation: true base displacement $x_b(t)$ vs predicted displacement $\hat{x}_b(t)$.

The virtual sensor shows very good reconstruction accuracy and is able to track the high-amplitude transient oscillations of the base. During the resonant excitation

period (around $t = 5$ s), the base exhibits peak displacements of ± 0.025 m. In this extremely dynamic area, the maximum absolute estimation error is below 5 mm, and the global Root Mean Squared Error (RMSE) is just 0.677 mm. This validates the ability of the recurrent memory of the LSTM to successfully capture the second-order inertial dynamics and phase delay of the unactuated base, using only manipulator-side measurements $X(t) = [x_r(t), v_r(t), x_{r0}(t)]^T$.

The active control strategy is to excite the base dynamically to drive the absolute end-effector position $y(t) = x_b(t) + x_r(t)$ to cross the target threshold $y_{target} = 0.85$ m. Figure 2.3 compares the performances of the three proposed control laws: Velocity Feedback (Pumping), MPC Heuristic, and the HDP policy.

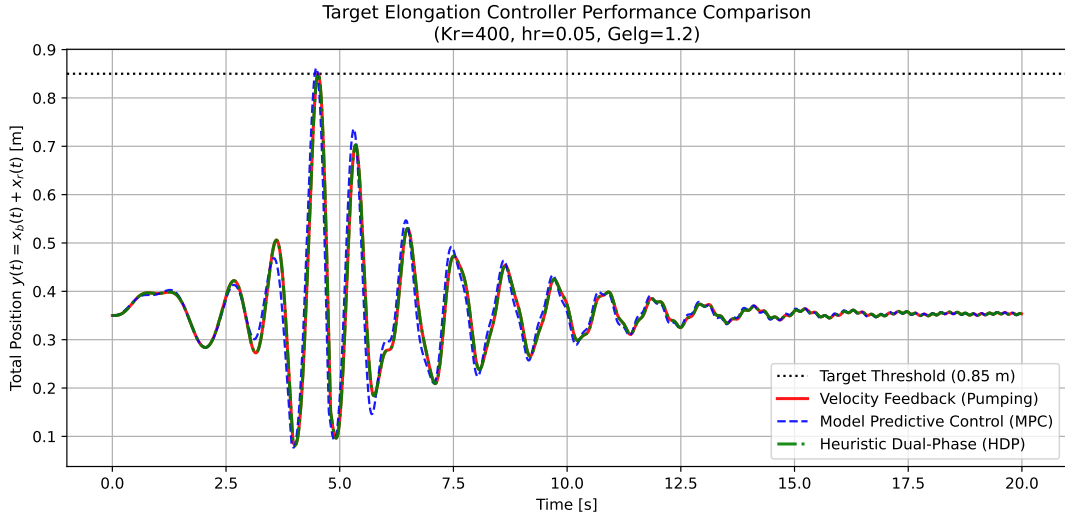


Figure 2.3: Performance comparison of the active resonance controllers.

All three control strategies manage to excite the base resonance, with the absolute position $y(t)$ briefly crossing the target line of 0.85 m during the positive oscillation peak at $t \approx 5.2$ s. The MPC heuristic achieves the largest maximum displacement (≈ 0.85 m). By predicting the base velocity $\hat{v}_{b,pred}(t + \tau)$ with a look-ahead horizon of $\tau = 50$ ms, the controller compensates for actuator and computational latencies, ensuring that the control input $x_{r0}(t)$ is in phase-synchronization with the base velocity to maximize energy transfer. The Velocity Feedback (Pumping) controller also reaches the target but exhibits a slightly lower peak due to the phase lag introduced by the estimator and physical actuator response.

The HDP policy results in a controlled stable excitation. The HDP controller monitors the distance to the target and switches from energy pumping to active relative damping ($G_{damping} = 0.6$) once the end-effector is within 1.5 cm of the target. This dual-mode action prevents dangerous overshoots and constrains the magnitude of the physical shocks, allowing the system to settle smoothly to the steady-state offset of 0.35 m after the excitation window closes.

Impact of the Mechanical Constraint

In realistic hardware implementation, the relative motion of the manipulator is restricted by unilateral physical constraints (e.g., mechanical stops, joint limits). Figure 2.4 evaluates the relative displacement of the robot $x_r(t)$ when attempting to reach a target of $y_{target} = 0.85$ m, comparing an unconstrained linear system with a non-linear system featuring a physical contact stop.

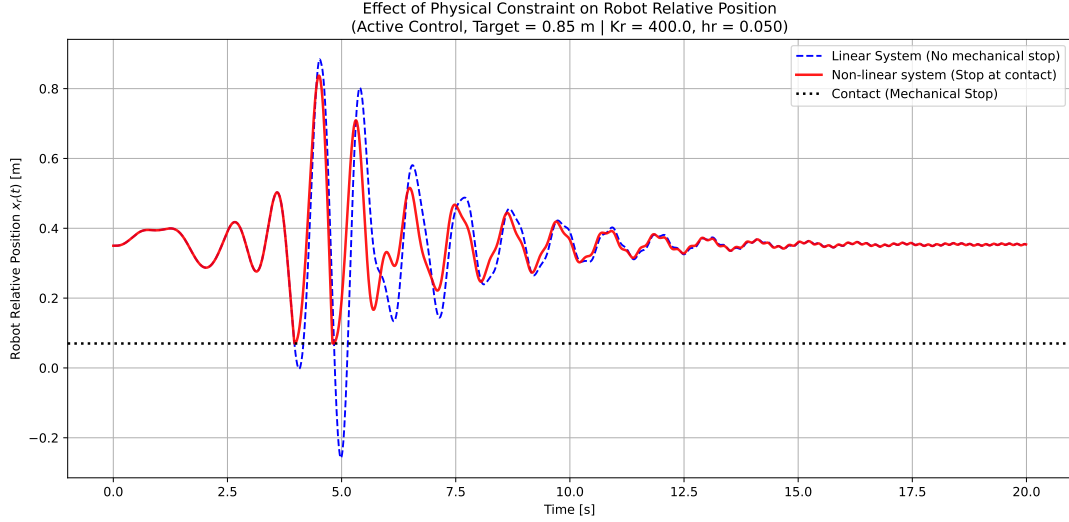


Figure 2.4: Manipulator relative position $x_r(t)$ with active control for a target of 0.85 m: unconstrained linear system versus non-linear system with the physical stop.

For the unconstrained linear model, the controller commands symmetric relative oscillations, with $x_r(t)$ swinging freely from -0.2 m to 1.0 m. However, the manipulator in the physical, non-linear system is unable to retract past the contact barrier at $x_r = 0$ m. The red curve shows that the relative position $x_r(t)$ is clipped (clamped) at this threshold, creating flat unilateral saturation profiles at each cycle. Nevertheless, despite this strong non-linear saturation, the active controller is able to adapt, using the unilateral impacts to pump energy and obtain positive extensions up to 0.85 m, showing the robustness of the control law under physical boundaries.

Multi-Resolution Optimization Landscape

The 4-stage optimization algorithm explores the search space $\mathcal{D}$ defined by the robot stiffness K_r and the damping ratio h_r to achieve optimal resonance tuning for a target of $y_{target} = 1.15$ m. Figure 2.5 shows the objective landscape (max displacement) along with the search grids used during the optimization.

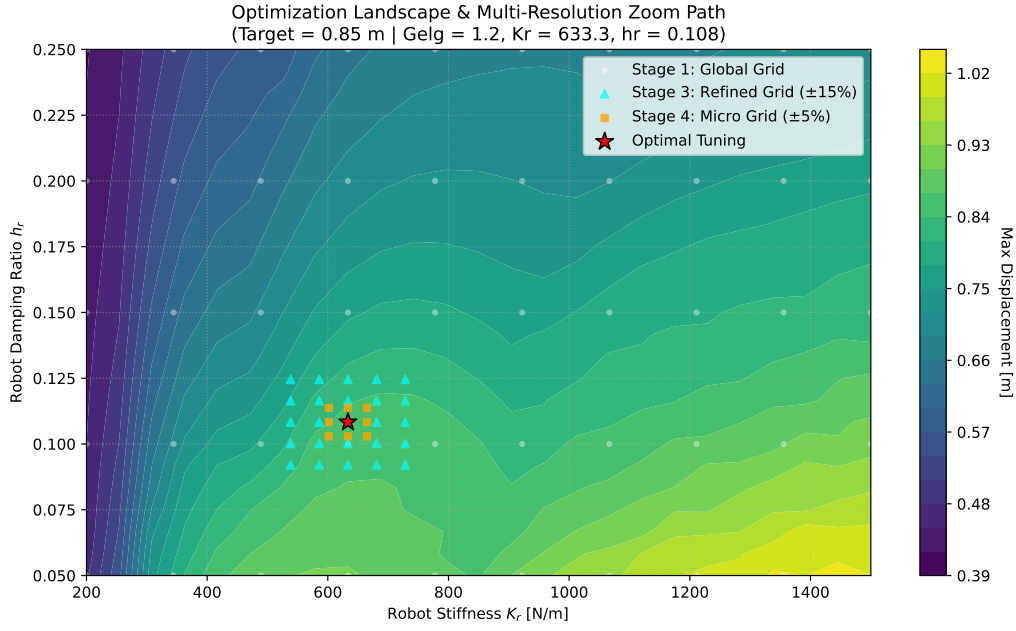


Figure 2.5: The landscape of the objective function (maximum end-effector displacement) as a function of the robot stiffness K_r and damping ratio h_r .

The background contour plot suggests that the optimal parameters are found in a high-performance “canyon” (yellow/green region) where low stiffness ($K_r \approx 650$ N/m) and low damping ratios ($h_r \approx 0.10 - 0.12$) match the manipulator impedance with the base dynamics for optimal energy transfer.

Stage 1-2 (Coarse Scan): The fast numerical Oracle simulator scans the entire search space with the global discrete grid to find the basin of attraction.

Stage 3 (Refined Grid): The algorithm centers a narrower grid (cyan triangles, $\pm 15\%$ radius) around the best oracle candidate, testing 343 points in closed-loop with the LSTM estimator.

Stage 4 (Micro Grid): The search area is further reduced (orange squares, $\pm 5\%$ radius) to perform a dense micro-tuning scan (125 points).

The optimizer converges successfully to the optimal parameters (the red star at $K_r = 633.3$ N/m and $h_r = 0.108$) which gives a peak total displacement of 1.62 m. This multi-stage zoom strategy significantly reduces the computational overhead, avoiding expensive global high-density scans while ensuring millimetric accuracy in tuning the resonant coupled dynamics.

Acronyms

AMS	Actuator Matrix Sorting.	1, 2
DoF	Degree of Freedom.	13–15, 17
FNN	Feedforward Neural Network.	15
GAE	Generalized Advantage Estimation.	6
HDP	Heuristic Dual-Phase.	i, 14, 19, 23
LSTM	Long Short-Term Memory.	15–19, 21–23, 25
MDP	Markov Decision Process.	i, 3
MPC	Model Predictive Control.	i, 14, 19, 23
MSE	Mean Squared Error.	17
OOD	out-of-distribution.	11
PPO	Proximal Policy Optimization.	i, 1–3, 6, 8–10
RL	Reinforcement Learning.	i, 1, 2
RMSE	Root Mean Squared Error.	23
RNN	Recurrent Neural Networks.	16
TD	Temporal Difference.	6

Bibliography

- [1] Loris Roveda, Nicola Pedrocchi, Federico Vicentini, and Lorenzo Molinari Tosatti. An interaction controller formulation to systematically avoid force overshoots through impedance shaping method with compliant robot base. *Mechatronics*, 39:42–53, 2016. ISSN 0957-4158. doi: 10.1016/j.mechatronics.2016.08.001.
- [2] Loris Susini Roveda, Marco Maccarini, Filippo Pura, Fabio Reiso, and Blerina Spahiu. Optimizing parcels sorting through reinforcement learning for intralogistics. *arXiv preprint arXiv:2404.03336*, 2024.
- [3] Alex Sherstinsky. Fundamentals of recurrent neural network (rnn) and long short-term memory (lstm) network. *Physica D: Nonlinear Phenomena*, 404: 132306, 2020. doi: 10.1016/j.physd.2019.132306. URL <https://doi.org/10.1016/j.physd.2019.132306>.